

Examic Evaluation System – Backend KT Guide

Models, Controllers & Cross-Collection Relationships

Table of Contents

- 1. [System Overview](#)
- 2. [Database Architecture](#)
- 3. [Evaluation Modes](#)
- 4. [Admin Models – osm_db](#)
- 5. [OSM Evaluation Models – osm_db](#)
- 6. [Scanner Models](#)
- 7. [CBT Models](#)
- 8. [Legacy / Utility Models – osm_db](#)
- 9. [OSM Controllers](#)
- 10. [Cross-Model Relationships \(implicit refs\)](#)
- 11. [Mermaid Diagram – Admin Models Only](#)
- 12. [Mermaid Diagram – Comprehensive System View](#)

1. System Overview

The Examic Evaluation System is a multi-mode academic examination management and evaluation platform. It supports three primary evaluation workflows:

Mode	What it is
OSM (OnScreen Script Marking)	Physical answer scripts are scanned → distributed to evaluators → marked online
Data Entry	Faculty manually enters marks for paper-based exams (no scanned scripts)
CBT (Computer-Based Testing)	Students take tests on a computer; subjective answers are evaluated online

The system is multi-tenant (multiple organizations/universities) and multi-campus.

2. Database Architecture

Four MongoDB databases are used. Each database has a dedicated Mongoose connection:

DB Variable	Purpose	Key Collections
osm_db	Core evaluation data — users, courses, exams, scripts, marks	users, courses, exam_informations, scanned_scripts, evaluation_results, cbt_marks, etc.
ORGANIZATION_DB	Organization-level sync data (answer script QR codes, test schedules for NU flow)	answer_script_qrcodes_nu, test_schedule_records
testplayer_db	CBT test-player data — question papers, user progress, answer scripts, schedules	questionpapers, userprogresses, answerscripts, test_schedule_records, users
scanner_db	Scanned script data for the Non-University (NU) scanner workflow	scanned_scripts_nu

Important: test_schedule_records exists in BOTH ORGANIZATION_DB and testplayer_db with slightly different schemas. Cross-database \$lookup is NOT possible in MongoDB — controllers handle cross-DB lookups via a **two-step approach** (fetch IDs from DB-A, then query DB-B separately).

3. Evaluation Modes

EXAM

- └ mode = "Paper Based + Data Entry" → Data Entry controller handles manual mark entry
- └ mode = "Paper Based + OSM" → Scripts scanned → distributed → marked in OSM
- └ mode = "Online / CBT" → Handled by CBT test-player; marks imported into CBT_Marks

`exam_informations.test_details[].mode` determines which flow applies per course within an exam.

4. Admin Models — `osm_db`

These models define the organizational and academic structure. They live in `osm_db` (except `testScheduleRecords` and `answerScriptQrCodesNuModel` which live in `ORGANIZATION_DB`).

4.1 `org_details` — Collection: `org_details`

File: `models/admin-models/orgDetailsModel.js`

Root entity for multi-tenancy. Every data record in the system carries an `org_id` pointing here.

Field	Type	Notes
<code>org_code</code>	String (unique)	Short identifier e.g. "NMIMS"
<code>org_name</code>	String (unique)	Full name
<code>org_type</code>	String	e.g. "University", "Institute"
<code>org_address</code>	String	Physical address
<code>privileges</code>	<code>[ObjectId] → privileges</code>	Modules/features this org has access to
<code>org_parent</code>	ObjectId	Self-referencing for multi-campus hierarchy (no Mongoose ref)
<code>qbms_id</code>	Number	External QBMS system reference
<code>org_location</code>	String	Campus location tag

Connections: Referenced by virtually every other model via `org_id`.

4.2 `privileges` — Collection: `privileges`

File: `models/admin-models/privilegesModel.js`

Defines the feature flags/module access for orgs and roles.

Field	Type	Notes
<code>p_name</code>	String	Human-readable name e.g. "OSM Module"
<code>p_key_name</code>	String	Key used in code checks
<code>p_id</code>	Number	Numeric ID for fast lookup

Connections: Referenced by `org_details.privileges[]` and `org_roles.privileges[]`.

4.3 `org_roles` — Collection: `org_roles`

File: `models/admin-models/orgRolesModel.js`

Custom roles defined per organization (e.g., "Head Evaluator", "Moderator").

Field	Type	Notes
name	String	Role name
description	String	What this role does
privileges	[ObjectId] → privileges	Permissions granted
org_id	ObjectId → org_details	Owning org

Connections: Assigned to faculty via `faculty_directories.role_details[]`.

4.4 org_admins — Collection: org_admins

File: `models/admin-models/orgAdminModal.js`

Admin user accounts (not faculty/students — these are org-level administrators who log in to manage the system).

Field	Type	Notes
email	String (unique)	Login email
password	String	Hashed
role	Number	Numeric role code
org_id	ObjectId → org_details	Owning org
isSuperAdmin	Boolean	Super admin flag
full_name	String	
phoneno	String	
gender	Enum: male/female/other	
dob	Date	
uniqueno	String (unique)	University serial number

4.5 users — Collection: users

File: `models/User.js`

Primary user entity for faculty and student accounts. **This is the most referenced collection in the system.**

Field	Type	Notes
full_name	String	
email	String (unique, lowercased)	
phone	String	
username	String (unique, lowercased)	
password	String	Hashed
dob	Date	
gender	String	

org_code Field	String Type	Organization code (denormalized) Notes
orgId	ObjectId	→ org_details._id (NO Mongoose ref , resolved in queries)
uniqueno	String (unique)	University roll number / employee ID
role	[ObjectId]	Array of org_roles._id (NO Mongoose ref)
type	Enum: admin/faculty	User category
system	Enum: OSM/TP/Scheduler	Which system the user belongs to

Connections:

- Enrolled students → student_enrolments.student_ids[]
- Faculty assignments → faculty_directories.user_details
- Script evaluators → distribution_rosters.evaluatorId , evaluation_results.evaluatorId
- Script owners → scanned_scripts.candidateId
- CBT marks → cbt_marks.candidateId (note: this candidateId points to testplayer_db.users , NOT osm_db.users)

4.6 schools — Collection: schools

File: models/admin-models/schoolsModel.js

Represents physical campuses or schools within an org.

Field	Type	Notes
name	String	Campus name
location	String	City/location tag
address	String	Full address
code	String	Short code
org_id	ObjectId → org_details	

Connections: yearly_masters.school → school where that batch is based.

4.7 programs — Collection: programs

File: models/admin-models/programsModel.js

Academic programs offered (e.g., MBA, B.Tech).

Field	Type	Notes
name	String	e.g. "B.Tech Computer Science"
code	String	e.g. "BTCS"
duration	Number	Years
org_id	ObjectId → org_details	
credits	String	Total credits
scheme	Enum: Annual/Trimester/Semester	Academic scheme

Connections: yearly_masters.program → which program that yearly master is for.

4.8 departments — Collection: departments

File: `models/admin-models/departmentsModel.js`

Academic departments (currently used for organization only; not heavily queried in OSM controllers).

Field	Type	Notes
<code>name</code>	String	
<code>code</code>	String	
<code>org_id</code>	ObjectId → <code>org_details</code>	

4.9 `semesters` — Collection: `semesters`

File: `models/admin-models/semestersModel.js`

Semester configuration per org. Lightly used in current controllers.

Field	Type	Notes
<code>no_of_semsisters</code>	String	Count of semesters
<code>location</code>	String	
<code>address</code>	String	
<code>org_id</code>	ObjectId → <code>org_details</code>	

4.10 `courses` — Collection: `courses`

File: `models/admin-models/coursesModel.js`

Individual courses/subjects.

Field	Type	Notes
<code>name</code>	String	Course name
<code>code</code>	String	e.g. "CS101"
<code>credits</code>	String	
<code>org_id</code>	ObjectId → <code>org_details</code>	

Connections:

- Listed in `yearly_masters.course_list[]`
 - Referenced by `exam_informations.test_details[].course_details`
 - Referenced by `faculty_directories.course_details`
 - Referenced by `distribution_rosters.courseId`, `evaluation_results.courseId`, `marking_pallets.courseId`
 - Referenced by `scanned_scripts.course_details`
-

4.11 `yearly_masters` — Collection: `yearly_masters`

File: `models/admin-models/yearlyMasterModel.js`

Central pivot entity. A yearly master represents a specific batch of students in a specific program at a specific school for a specific academic year and term.

Field	Type	Notes
<code>school</code>	ObjectId → <code>schools</code>	Campus
<code>program</code>	ObjectId → <code>programs</code>	Academic program

batch Field	String Type	e.g. "2021-2025" Notes
batch_name	String	Friendly name
academic_year	String	e.g. "2023-24"
term	String	e.g. "Semester 3"
course_list	[ObjectId] → courses	Courses offered this term
org_id	ObjectId → org_details	

Connections:

- student_enrolments.academic_details → which batch students are enrolled in
- faculty_directories.academic_details → which batch faculty are assigned to
- exam_informations.yearly_master_details → exam belongs to this batch
- exam_informations.multi_yearly_masters[] → exam shared across multiple batches (multi-campus)
- exam_time_tables.academic_details → timetable for this batch
- CBT_Schema/testSchedule.yearly_master_id → CBT schedule linked to a batch

4.12 student_enrolments — Collection: student_enrolments

File: models/admin-models/studentEnrolmentModel.js

Maps a batch (yearly master) to the students enrolled in it.

Field	Type	Notes
academic_details	ObjectId → yearly_masters	Batch
org_id	ObjectId → org_details	
student_ids	[ObjectId]	→ users._id (NO Mongoose ref)

Usage in code: studentEnrolmentModel.findOne({ academic_details: yearlyMasterId }) then User.find({ _id: { \$in: enrolment.student_ids } }) to get student details.

4.13 faculty_directories — Collection: faculty_directories

File: models/admin-models/facultyDirectoryModel.js

Assignment of faculty members to courses within an academic batch.

Field	Type	Notes
academic_details	ObjectId → yearly_masters	Batch context
role_details	[ObjectId] → org_roles	Roles assigned
course_details	ObjectId → courses	Which course
user_details	ObjectId	→ users._id (NO Mongoose ref)
org_id	ObjectId → org_details	

Used in controllers to:

- Find eligible evaluators for a course (getEvaluatorsForCourse)
- Find a faculty member's assigned courses (getDataEntryTests, getICAssessments)
- Find alternate moderators (applyModerationAndCreateRosters)

4.14 faculty_evaluation_rosters — Collection: faculty_evaluation_rosters

File: `models/admin-models/facultyEvaluationRostersModel.js`

Note: This model was an earlier design for evaluation assignment. It has been superseded by `DistributionRosterModel` in current controllers. It is not actively used in any OSM controller but the schema exists.

Field	Type	Notes
<code>academic_details</code>	<code>ObjectId</code> → <code>yearly_masters</code>	
<code>exam_details</code>	<code>ObjectId</code> → <code>exam_informations</code>	
<code>user_details</code>	<code>[ObjectId]</code>	→ <code>users._id</code> (NO ref)
<code>course_details</code>	<code>ObjectId</code> → <code>courses</code>	
<code>org_id</code>	<code>ObjectId</code> → <code>org_details</code>	
<code>duration.start/end</code>	Date	Evaluation window

4.15 `exam_types` — Collection: `exam_types`

File: `models/admin-models/examTypesModel.js`

Top-level category for exams.

Field	Type	Notes
<code>type</code>	String	e.g. "Internal Assessment", "End Term Examination", "Supplementary"

Usage: `examTypesModel.findOne({ type: "Internal Assessment" })` in IC assessment controllers to filter exams.

4.16 `exam_sub_types` — Collection: `exam_sub_types`

File: `models/admin-models/examSubTypes.js`

Sub-categories under an exam type.

Field	Type	Notes
<code>type</code>	String	e.g. "First Internal", "Mid Term"
<code>exam_type</code>	<code>ObjectId</code> → <code>exam_types</code>	Parent type

4.17 `test_types` — Collection: `test_types`

File: `models/admin-models/testTypesModel.js`

Type of assessment test within an exam (e.g., the component type).

Field	Type	Notes
<code>type</code>	String	e.g. "CT1", "Assignment", "Lab"

Referenced by: `exam_informations.test_details[].test_type` and `exam_time_tables.course_schedule_details[].type`.

4.18 `exam_informations` — Collection: `exam_informations`

File: `models/admin-models/examInformationsModel.js`

The central exam configuration entity. Each document describes one exam session (e.g., "End Term Exam 2024 – Semester 3"). Contains an embedded array of per-course test details.

Field	Type	Notes
exam_name	String	
exam_type	ObjectId → exam_types	
exam_sub_type	ObjectId → exam_sub_types	Optional
yearly_master_details	ObjectId → yearly_masters	Which batch this exam is for
test_details	[testDetailsSchema]	Embedded array — one entry per course
org_id	ObjectId	→ org_details._id (no explicit ref)
moderationRuleSetId	ObjectId → moderation_rule_sets	Linked moderation rules
is_multi_campuses	Boolean	Whether this exam spans campuses
is_osm_enabled	Boolean	Whether OSM is enabled for this exam
multi_yearly_masters	[ObjectId] → yearly_masters	Linked batches for multi-campus exams

Embedded test_details sub-schema:

Field	Type	Notes
test_type	ObjectId → test_types	Component type
blueprint_details.blueprint_url	String	Azure Blob URL for blueprint PDF
blueprint_details.question_paper_url	String	Azure Blob URL for question paper PDF
course_details	ObjectId → courses	Which course
marks	String	Total marks for this course
mode	String	"Paper Based + Data Entry" / "Paper Based + OSM" / "Online"
weightage	Number	Weightage percentage
duration	Number	Duration in minutes
exam_group	String	Grouping label

Key concept: test_details._id (the auto-generated subdocument _id) is used throughout the system as testId in distribution_rosters, evaluation_results, and marking_pallets. There is no separate tests collection — these subdocument IDs are the test identifiers.

4.19 exam_time_tables — Collection: exam_time_tables

File: models/admin-models/examTimeTableModel.js

Timetable for an exam — maps courses to their scheduled dates/times.

Field	Type	Notes
academic_details	ObjectId → yearly_masters	
course_schedule_details	[courseScheduleDetails]	Per-course schedule
exam_id	ObjectId → exam_informations	
org_id	ObjectId	

Embedded `courseScheduleDetails`:

Field	Type
<code>course_id</code>	ObjectId → <code>courses</code>
<code>date</code>	Date
<code>from_time</code>	Date
<code>to_time</code>	Date
<code>type</code>	ObjectId → <code>test_types</code>
<code>venue</code>	[ObjectId] (venue refs, no formal model)

4.20 `time_table_rosters` — Collection: `time_table_rosters`

File: `models/admin-models/timeTableRosterModel.js`

Maps individual students to their timetable (which exam rooms/venues they appear in).

Field	Type	Notes
<code>student_id</code>	ObjectId	→ <code>users._id</code> (no ref)
<code>exam_id</code>	ObjectId → <code>exam_informations</code>	
<code>yearly_master_id</code>	ObjectId → <code>yearly_masters</code>	
<code>time_table_id</code>	ObjectId → <code>exam_time_tables</code>	
<code>course_details</code>	[<code>courseDetailSchema</code>]	Per-course schedule including venue

4.21 `marking_pallets` — Collection: `marking_pallets`

File: `models/admin-models/MarkingPalletModel.js`

Defines the question-level marking scheme for a specific exam/course/test combination. Evaluators use this to know max marks per question.

Field	Type	Notes
<code>examId</code>	ObjectId → <code>exam_informations</code>	
<code>courseId</code>	ObjectId → <code>courses</code>	
<code>testId</code>	ObjectId	→ <code>exam_informations.test_details._id</code> (no formal ref)
<code>org_id</code>	ObjectId → <code>org_details</code>	
<code>status</code>	Enum: <code>draft/frozen</code>	Once frozen, cannot be modified
<code>totalMarks</code>	Number	Sum of all question max marks
<code>allowNegativeMarking</code>	Boolean	
<code>questions</code>	[<code>questionSchema</code>]	Array of question definitions
<code>createdBy</code>	ObjectId → <code>users</code>	
<code>frozenAt</code>	Date	

<code>frozenBy</code> Field	<code>ObjectId</code> → <code>users</code> Type	Notes
--------------------------------	--	-------

Embedded `questionSchema`:

Field	Type	Notes
<code>questionNo</code>	String	e.g. "1a", "2b"
<code>label</code>	String	Display label
<code>maxMarks</code>	Number	Max marks for this question
<code>cobo</code>	String	Course Outcome / Bloom's Objective e.g. "c01"

Unique index: (`examId`, `courseId`, `testId`) — one pallet per exam/course/test combo.

Propagation: When a pallet is saved for a multi-campus exam (`multi_yearly_masters`), the controller automatically propagates (upserts) it to all linked exams.

4.22 `sync_logs` — Collection: `sync_logs`

File: `models/admin-models/syncLogModel.js`

Audit trail for data synchronization operations between main DB and organization-specific databases.

Field	Type	Notes
<code>triggeredBy.userId</code>	<code>ObjectId</code> → <code>users</code>	Who triggered the sync
<code>triggeredBy.userName</code>	String	Denormalized name
<code>status</code>	Enum: <code>COMPLETED</code> / <code>PARTIAL_SUCCESS</code> / <code>FAILED</code>	
<code>summary</code>	Object	Counts: <code>totalRequested</code> , <code>totalFound</code> , <code>successful</code> , <code>failed</code>
<code>performance</code>	Object	Timing: <code>totalTimeMs</code> , <code>avgTimePerOrgMs</code> , <code>maxConcurrency</code>
<code>results</code>	[Object]	Per-org sync results with collection-level detail

4.23 `answer_script_qrcodes_nu` — Collection: `answer_script_qrcodes_nu` (ORGANIZATION_DB)

File: `models/admin-models/answerScriptQrCodesNuModel.js`

Maps QR codes printed on Non-University answer scripts to their test schedule records.

Field	Type	Notes
<code>qrcode</code>	String (unique)	The QR code value
<code>test_id</code>	<code>ObjectId</code> → <code>test_schedule_records</code>	Which test
<code>schedule_id</code>	<code>ObjectId</code> → <code>test_schedule_records</code>	Which schedule
<code>candidate_id</code>	<code>ObjectId</code>	→ <code>users._id</code> (no ref)

4.24 `test_schedule_records` (ORGANIZATION_DB) — Collection: `test_schedule_records`

File: `models/admin-models/testScheduleRecords.js`

Test schedule records synced to the organization database. Used in the NU (Non-University) scanner QR code workflow.

Field	Type	Notes
-------	------	-------

Field	String (unique) Type	Human-readable schedule identifier Notes
exam_mode	Number	
test_schedule_info	Object {name, description}	
users_list	[ObjectId]	→ users._id (students enrolled, stored as ObjectIds in ORG_DB)
selected_test	[{test_name, test_code, duration}]	
start_timestamp	Date	
end_timestamp	Date	
orgId	ObjectId	
instruction	String	
proctors_list	[ObjectId] → users	
is_proctoring_enabled	Boolean	
is_ai_proc	Boolean	
is_human_physical	Boolean	
is_human_proc	Boolean	
evaluators	[{test_name, test_code, evaluatorsList: [ObjectId]}]	

5. OSM Evaluation Models — osm_db

These three models are the core of the script evaluation workflow.

5.1 Distribution_Roster — Collection: distribution_rosters

File: models/osm-models/DistributionRosterModel.js

Represents an assignment of a set of scanned scripts to a specific evaluator for a specific evaluation task. **This is the primary workflow tracking entity for OSM.**

Field	Type	Notes
evaluatorId	ObjectId → users	The evaluator assigned
examId	ObjectId → exam_informations	
courseId	ObjectId → courses	
testId	ObjectId	→ exam_informations.test_details._id (no formal ref)
evaluationType	Enum: primary/review/moderation/re_evaluation	Round of evaluation
previousRosterId	ObjectId → Distribution_Roster	For review rosters: link to primary roster
assignedScripts	[ObjectId] → Scanned_Scripts	The scripts to evaluate
scriptCount	Number	Denormalized count of assignedScripts
status	Enum: pending/in_progress/completed/reassigned	Roster lifecycle
evaluationDueDate	Date	

Field	Type	Notes
evaluationRound	Number	1 = primary, 2 = review, etc.
assignedBy	ObjectId → users	Admin who created this assignment
org_id	ObjectId → org_details	
notes	String	Auto-generated notes (e.g., creation timestamp)

Key compound indexes:

- (examId, courseId, testId) — find all rosters for a specific test
- (previousRosterId) — find review rosters for a primary one

Status lifecycle:

pending → in_progress (when first script is saved) → completed (when all scripts done)

5.2 Evaluation_Result — Collection: evaluation_results

File: models/osm-models/EvaluationResultModel.js

Stores the marks and annotations for a single evaluation instance. Dual-mode: supports both OSM (script-based) and Data Entry (student-based).

Field	Type	Notes
org_id	ObjectId → org_details	
entryType	Enum: osm/data_entry	Which workflow
scriptId	ObjectId → Scanned_Scripts	OSM only — the scanned script
rosterId	ObjectId → Distribution_Roster	OSM only — the assignment
studentId	ObjectId → users	Data Entry only — the student
examId	ObjectId → exam_informations	Data Entry only
courseId	ObjectId → courses	Data Entry only
testId	ObjectId	Data Entry only — exam_informations.test_details._id
evaluatorId	ObjectId → users	Who marked it
totalMarksAwarded	Number	Total marks given
marksBreakdown	[{questionNumber: String, marksAwarded: Number}]	Per-question marks
annotations	String	JSON string of canvas annotations
status	Enum: not_started/in_progress/completed	
overallComment	String	Evaluator's general comment

Unique partial indexes:

- OSM mode: (scriptId, rosterId) unique where entryType = "osm"
- Data Entry mode: (studentId, examId, courseId, testId) unique where entryType = "data_entry"

5.3 moderation_rule_sets — Collection: moderation_rule_sets

File: models/osm-models/ModerationRuleSetModel.js

Defines automated rules for triggering second-round (moderation) evaluation.

Field	Type	Notes
-------	------	-------

name Field	String (unique) Type	e.g. "High/Low Scorer Policy" Notes
org_id	ObjectId → org_details	
rules	[ruleSchema]	Array of rules
isActive	Boolean	

Embedded ruleSchema :

Field	Type	Notes
type	Enum: MARK_PERCENTAGE / DEVIATION_FROM_AVERAGE / RANDOM_SAMPLING	
condition	Enum: GREATER_THAN / LESS_THAN / BETWEEN / PERCENTAGE_OF_TOTAL	
values	[Number]	Threshold values
name	String	e.g. "High Scorers"

How it works:

- Admin links a rule set to an exam via exam_informations.moderationRuleSetId
- After primary evaluation completes, admin calls applyModerationAndCreateRosters
- Controller evaluates each completed result against the rules
- Scripts meeting criteria get new distribution_rosters created with evaluationType = "moderation"

6. Scanner Models

6.1 Scanned_Scripts — Collection: scanned_scripts (osm_db)

File: models/scanner-models/ScriptModel.js

The main scanned script entity for the standard OSM workflow. Each document represents one student’s scanned answer script for one exam/course combination.

Field	Type	Notes
studentUniqueNo	String	Student roll number (denormalized)
uniqueScriptCode	String	Unique 16-char code printed/scanned on the script
candidateId	ObjectId → users	The student
exam_details	ObjectId → exam_informations	
test_details	ObjectId	→ exam_informations.test_details._id (no ref)
course_details	ObjectId → courses	
org_id	ObjectId → org_details	
pages	[[{pageNumber, type: left/right, timestamp}]]	Individual page metadata
pdfPath	String	Azure Blob path to the combined PDF
pageCount	Number	
version	Number	Script version (for re-scans)
uploadStatus	Enum: initialized/processing/complete/error	
scannedBy	ObjectId → users	Who operated the scanner
verificationStatus	Enum: unverified/verified/flagged	QA status

Key virtual: `scriptId` → `this._id` (alias for the `_id` field)

6.2 Scanned_Scripts_NU — Collection: `scanned_scripts_nu` (`scanner_db`)

File: `models/scanner-models/ScannedScriptNUModel.js`

Scanned scripts for the Non-University (external/partner institution) flow. Stored in a separate `scanner_db`. Similar structure to `Scanned_Scripts` but references NU-specific fields.

Field	Type	Notes
<code>studentUniqueNo</code>	String	
<code>uniqueScriptCode</code>	String (unique)	
<code>candidateId</code>	ObjectId	→ some candidate (no formal ref)
<code>test_details</code>	ObjectId → <code>test_schedule_records</code>	Points to ORGANIZATION_DB
<code>schedule_details</code>	ObjectId → <code>test_schedule_records</code>	Points to ORGANIZATION_DB
<code>org_id</code>	ObjectId → <code>org_details</code>	
<code>pages</code>	[<code>{pageNumber, type, timestamp, extension}</code>]	
<code>pdfPath</code>	String	
<code>pageCount</code>	Number	
<code>uploadStatus</code>	Enum: <code>initialized/processing/complete/error</code>	
<code>scannedBy</code>	ObjectId → <code>users</code>	
<code>verificationStatus</code>	Enum: <code>unverified/verified/flagged</code>	

6.3 Unique_Script_Code_Mapping — Collection: `unique_script_code_mappings` (`osm_db`)

File: `models/scanner-models/UniqueScriptCodeMappingModel.js`

Pre-generated mapping of unique script codes to candidates. Used to link a scanned script code back to the student before the script is uploaded.

Field	Type	Notes
<code>uniqueScriptCode</code>	String (unique)	
<code>candidateId</code>	ObjectId	→ <code>users._id</code> (no ref)
<code>testId</code>	ObjectId	→ <code>exam_informations.test_details._id</code> (no ref)
<code>examId</code>	ObjectId	→ <code>exam_informations._id</code> (no ref)
<code>courseId</code>	ObjectId	→ <code>courses._id</code> (no ref)

7. CBT Models

CBT models span two databases: `testplayer_db` (the student-facing test platform) and `osm_db` (where evaluation results are stored).

7.1 CBT_Marks — Collection: `cbt_marks` (`osm_db`)

File: `models/CBT_Schema/CBTMarks.js`

Stores evaluation results for CBT (online) tests. Supports both objective auto-evaluation and subjective rubrics-based evaluation.

Field	Type	Notes
<code>scriptId</code>	ObjectId	→ <code>AnswerScript._id</code> in <code>testplayer_db</code> (cross-DB, no ref)
<code>candidateId</code>	ObjectId	→ <code>testplayer_db.User._id</code> (cross-DB, no ref)
<code>testId</code>	ObjectId	→ <code>testplayer_db.QuestionPaper._id</code> (cross-DB, no ref)
<code>testCode</code>	Mixed	Test code(s)
<code>schedule_Id</code>	ObjectId	→ <code>testplayer_db.test_schedule_records._id</code> (cross-DB, no ref)
<code>test_type</code>	String	
<code>test_sub_type</code>	String	
<code>is_hiring_assessment</code>	Boolean	
<code>marks</code>	[markSchema]	Per-question marks array
<code>totalMarks</code>	Number	Max marks
<code>totalMarksObtained</code>	Number	Actual marks
<code>isObjectiveEvaluated</code>	Boolean	
<code>isSubjectiveEvaluated</code>	Boolean	
<code>isLongEvaluated</code>	Boolean	
<code>isReportGenerated</code>	Boolean	Report published to student
<code>isRubricsEvaluated</code>	Boolean	For rubric-based subjective questions
<code>org_id</code>	ObjectId → <code>org_details</code>	
<code>examInfoId</code>	ObjectId → <code>exam_informations</code>	Optional link to OSM exam
<code>courseId</code>	ObjectId → <code>courses</code>	Optional link to course

Embedded `markSchema` per question:

Field	Type
<code>question</code>	String (question ID)
<code>questionText</code>	Mixed
<code>alloted_mark</code>	Number
<code>type</code>	String (objective/subjective/long)
<code>is_rubrics_evaluated</code>	Boolean
<code>max_marks</code>	Number
<code>correct_answer</code>	Mixed
<code>user_answer</code>	Mixed
<code>rubrics_evaluation</code>	[rubricSchema]

7.2 CBT_Script_Annotation — Collection: cbt_script_annotations (osm_db)

File: models/CBT_Schema/CBTScriptAnnotations.js

Annotations made by evaluators on CBT answer images (for subjective questions).

Field	Type	Notes
scriptId	ObjectId	→ AnswerScript._id in testplayer_db (cross-DB, no ref)
annotations	[questionAnnotationsSchema]	Per-question, per-image annotations
lastUpdated	Date	

7.3 CBT_Marks_NU — Collection: cbt_marks_nu (osm_db)

File: models/CBT_Schema/CBT_Marks_NU.js

Marks for the Non-University CBT flow (separate from main CBT marks).

Field	Type	Notes
scriptId	ObjectId (unique)	→ Scanned_Scripts_NU._id (cross-DB, no ref)
rosterId	ObjectId	The schedule_id from the NU flow
evaluatorId	ObjectId	→ users._id (no ref)
marksBreakdown	[{questionNumber, marksAwarded}]	
overallComment	String	
annotations	String	JSON string from Konva canvas
status	Enum: in_progress/completed	

7.4 AnswerScript — Collection: answerscripts (testplayer_db)

File: models/CBT_Schema/AnswerScript.js

Encrypted answer submission from a CBT candidate. Contains encrypted question+answer data.

Field	Type	Notes
testCode	String	Which test
schedule_Id	ObjectId → test_schedule_records	Within testplayer_db
uniqueno	String	Student roll number
encryptedData	String	AES-encrypted answers
encryptedAesKey	String	Encrypted AES key
timeTaken	Number	Seconds
iv / signature / merkleRoot	String	Encryption integrity fields
isSubjectiveAnswered	Boolean	
isLongAnswered	Boolean	

Field	Type	Notes
isObjectiveAnswered	Boolean	
isObjectiveEvaluated	Boolean	
isSubjectiveEvaluated	Boolean	
isLongEvaluated	Boolean	
canViewReport	Boolean	
testAttempt	Number	Attempt number
candidateId	ObjectId → User	Within testplayer_db
test_Id	ObjectId → QuestionPaper	Within testplayer_db

7.5 QuestionPaper — Collection: questionpapers (testplayer_db)

File: models/CBT_Schema/QuestionPaper.js

Encrypted question paper stored in the test player database.

Field	Type	Notes
test_name	String	
unit	String	
course_code	String	
course_name	String	
test_code	[String] (unique)	One or more test codes
test_type	String	
test_sub_type	String	
org_code	String	Org identifier
isAdaptive	Boolean	Adaptive testing
isRubricsPresent	Boolean	Whether subjective rubrics exist
encryptedData/encryptedAesKey/iv/signature/merkleRoot	String	Encryption fields
(Many boolean flags)	Boolean	isShuffle, isManualEval, isMLEval, containsSubjective, etc.
duration	Number	Minutes
auto_evaluate	Boolean	

7.6 QuestionPaperWithAnswer — Collection: questionpaperanswerscripts (testplayer_db)

File: models/CBT_Schema/QuestionPaperWithAnswer.js

Same structure as QuestionPaper but also contains the answer key. Used internally for evaluation.

7.7 UserProgress — Collection: userprogresses (testplayer_db)

File: `models/CBT_Schema/UserProgress.js`

Tracks a student's real-time progress during a CBT test.

Field	Type	Notes
<code>userId</code>	ObjectId → <code>User</code>	testplayer_db user
<code>testCode</code>	String	
<code>schedule_Id</code>	ObjectId → <code>test_schedule_records</code>	
<code>currentQuestionIndex</code>	Number	
<code>remainingTime</code>	Number	
<code>breaksTaken</code>	Number	
<code>isSubmitted</code>	Boolean	True = student has submitted
<code>isAnswerScriptGenerated</code>	Boolean	
<code>timeTaken</code>	Number	
<code>answers</code>	Array	Student answers
<code>flagged</code>	Array	Flagged questions
<code>fileUrls</code>	Array	Uploaded file URLs
<code>test_Id</code>	ObjectId → <code>QuestionPaper</code>	

7.8 `test_schedule_records` (testplayer_db) – Collection: `test_schedule_records`

File: `models/CBT_Schema/testSchedule.js`

CBT test schedules as seen by the test player. Similar to the ORGANIZATION_DB version but with additional fields for OSM linking.

Field	Type	Notes
<code>schedule_id</code>	String (unique)	
<code>exam_mode</code>	Number	
<code>test_schedule_info</code>	{ <code>name</code> , <code>description</code> }	
<code>users_list</code>	[<code>String</code>]	Usernames/uniquenos (NOT ObjectIds – different from ORGANIZATION_DB version)
<code>selected_test</code>	[{ <code>test_name</code> , <code>test_code</code> , <code>duration</code> , <code>exam_info</code> , <code>course_details</code> }]	Has extra fields: <code>exam_info</code> (→ <code>osm_db exam_informations._id</code>) and <code>course_details</code> (→ <code>osm_db courses._id</code>)
<code>start_timestamp</code> / <code>end_timestamp</code>	Date	
<code>orgId</code>	ObjectId	
<code>instruction</code>	String	
<code>proctors_list</code>	[<code>ObjectId</code>] → <code>users</code>	
<code>is_proctoring_enabled</code>	Boolean	
<code>yearly_master_id</code>	ObjectId	→ <code>osm_db yearly_masters._id</code> (cross-DB, no ref)

Field	Type	Notes
course_id	ObjectId	→ osm_db courses._id (cross-DB, no ref)
exam_info_id	ObjectId	→ osm_db exam_informations._id (cross-DB, no ref)

Critical difference from ORGANIZATION_DB version: `users_list` here stores **strings** (usernames/uniquenos) not ObjectIds. This is a known discrepancy handled in controllers.

7.9 User (testplayer_db) – Collection: users (testplayer_db)

File: `models/CBT_Schema/User.js`

CBT candidate accounts in the test player database. Separate from `osm_db.users`.

Field	Type	Notes
full_name	String	
email	String (unique)	
phone	String	
username	String (unique)	
password	String	
dob	Date	
gender	String	
uniqueno	String (unique)	University roll number
role	Enum: admin/candidate	

8. Legacy / Utility Models — osm_db

These models exist from an earlier architecture. They are **not actively used** by the current OSM controllers but are in the codebase.

Model	Collection	Notes
Exam (<code>models/Exam.js</code>)	exams	Simple exam with just a name
Script (<code>models/Script.js</code>)	scripts	Basic script with files array; replaced by <code>Scanned_Scripts</code>
ScriptAnnotations (<code>models/ScriptAnnotations.js</code>)	script_annotations	Legacy annotation model; replaced by <code>Evaluation_Result.annotations</code>
Mark (<code>models/Mark.js</code>)	marks	Simple marks per script; replaced by <code>Evaluation_Result</code>
MarkedScript (<code>models/MarkedScript.js</code>)	marked_scripts	Marked script with files; not used in current flow

9. OSM Controllers

All controllers are in `controllers/OSM/`. They are Express route handlers using `async/await`.

9.1 adminRosterController.js

Handles admin-side roster creation, moderation workflow, and engine data endpoints.

Function	Route	Description
----------	-------	-------------

Function	Route	Description
<code>getOverallRoster</code>	POST <code>/api/osm/admin/rosters/overall-evaluation-roster</code>	High-level view of all evaluators, their assignments, and completion counts grouped by evaluator
<code>createAutomatedRosters</code>	POST <code>/api/osm/admin/rosters/create-automated</code>	Creates primary distribution rosters from an admin-provided allocation plan. Also links <code>moderationRuleSetId</code> to selected exams
<code>getAllScannedPapers</code>	POST <code>/api/evaluation/papers/all</code>	Paginated + filterable list of all scanned scripts for an org
<code>getAllProgramsWithMetadata</code>	POST <code>/api/osm/admin/rosters/engine-data/programs-all</code>	Programs with their yearly masters and campus metadata – for the roster allocation UI
<code>getExamsForEngine</code>	POST <code>/api/osm/admin/rosters/engine-data/exams</code>	Exams with unassigned script counts; supports filtering by <code>programIds</code> ; groups linked exams
<code>getProgramsForExams</code>	POST <code>/api/osm/admin/rosters/engine-data/programs</code>	Programs associated with specific exams (via <code>yearly_masters</code>)
<code>getCoursesForEngine</code>	POST <code>/api/osm/admin/rosters/engine-data/courses</code>	Courses within selected exams, with script counts
<code>getEvaluatorsForCourse</code>	POST <code>/api/osm/admin/rosters/engine-data/evaluators</code>	Faculty eligible to evaluate a specific course (from <code>faculty_directories</code>)
<code>applyModerationAndCreateRosters</code>	POST <code>/api/osm/moderation/apply-rules</code>	Applies moderation rules to completed primary evaluations → creates moderation rosters
<code>getRuleSetsForOrg</code>	POST <code>/api/osm/moderation/get-rule-sets</code>	Active moderation rule sets for an org
<code>getCompletedExamsForModeration</code>	POST <code>/api/osm/admin/exams/completed-for-moderation</code>	Exams where all primary rosters are complete and moderation hasn't been applied yet
<code>getCoursesForExam</code>	POST <code>/api/osm/admin/exams/get-courses-for-exam</code>	Courses listed in a specific exam's <code>test_details</code>
<code>getHierarchicalRosterData</code>	POST <code>/api/osm/admin/rosters/hierarchical</code>	Hierarchical view: Exam → Program → Course → Rosters (with completion stats)

Key patterns:

- Uses `DistributionRosterModel.distinct("assignedScripts", ...)` to track which scripts are already assigned
- Multi-campus exam support: groups exams by `exam_name` + `multi_yearly_masters` in `getExamsForEngine`
- Moderation: finds eligible moderator from `faculty_directories` who is **different** from the primary evaluator

9.2 `dataEntryController.js`

Handles the Data Entry evaluation mode – where faculty manually enters marks without scanned scripts.

Function	Route	Description
<code>getDataEntryTests</code>	POST <code>/api/evaluation/data-entry/tests</code>	Gets all data-entry-mode tests assigned to a faculty. Supports <code>purpose = "data-entry"</code> (mode filter) or <code>"ica"</code> (Internal Assessment filter). Returns hierarchical: Year → Program → Course
<code>getStudentsForTest</code>	POST <code>/api/evaluation/data-entry/students</code>	Gets all enrolled students for a specific test with their existing marks and the marking pallet
<code>saveDataEntryMarks</code>	POST <code>/api/evaluation/data-entry/save-marks</code>	Upserts <code>Evaluation_Result</code> for a single student (<code>entryType = "data_entry"</code>)
<code>saveDataEntryMarksBulk</code>	POST <code>/api/evaluation/data-entry/save-marks-</code>	Bulk upserts marks for multiple students using <code>bulkWrite</code>

Function	bulk Route	Description
----------	---------------	-------------

Key patterns:

- Uses `facultyDirectoryModel` to filter which courses a faculty is assigned to
- Uses `studentEnrolmentModel` to get the student list for a batch
- Uses `MarkingPalletModel` to return the question-level marking scheme alongside student list
- `EvaluationResultModel.findOneAndUpdate({ entryType: "data_entry", examId, courseId, testId, studentId }, ..., { upsert: true })`

9.3 `evaluationAdminController.js`

Admin tools for configuring the evaluation setup: synoptic uploads and marking pallets.

Function	Route	Description
<code>getFilterOptions</code>	POST <code>/api/evaluation/admin/filter-options</code>	Programs, academic years, and exam types for admin filter UI
<code>getAdminCoursesWithStatus</code>	POST <code>/api/evaluation/admin/courses-with-status</code>	Courses with synoptic upload status and marking pallet status for admin console
<code>uploadSynoptics</code>	POST <code>/api/evaluation/admin/synoptics/upload</code>	Uploads blueprint/question paper PDF to Azure Blob Storage, updates <code>exam_informations.test_details[].blueprint_details</code> , propagates to linked exams
<code>getMarkingPallet</code>	POST <code>/api/evaluation/admin/marking-pallet/get</code>	Fetches pallet for org/exam/course/test
<code>saveMarkingPallet</code>	POST <code>/api/evaluation/admin/marking-pallet/save</code>	Creates or updates marking pallet; handles <code>freeze</code> flag; propagates to linked exams
<code>getSynopticSasUrl</code>	POST <code>/api/evaluation/admin/synoptics/sas-url</code>	Generates time-limited Azure SAS URL for a stored blob

Key patterns:

- **Multi-campus propagation:** When an exam has `multi_yearly_masters`, saving a pallet or synoptic automatically propagates (upserts) it to all linked exams sharing the same course
- Marking pallets are frozen by setting `status = "frozen"` — once frozen, further updates are rejected unless re-freeze is requested

9.4 `evaluationDashboardController.js`

The main evaluation dashboard — used by both faculty evaluators and admins.

Function	Route	Description
<code>getPrimaryRosters</code>	GET <code>/api/evaluation/rosters/primary</code>	All primary rosters assigned to logged-in evaluator (with completion counts)
<code>getRosterDetails</code>	POST <code>/api/evaluation/roster-details</code>	Scripts list within a roster with their individual evaluation status
<code>getGradingData</code>	GET <code>/api/evaluation/script/:uniqueScriptCode</code>	Full grading page data: script PDF (SAS URL), marking pallet, saved progress
<code>saveEvaluationProgress</code>	POST <code>/api/evaluation/save-progress</code>	Saves marks and annotations; auto-updates roster status to <code>in_progress</code> / <code>completed</code>
<code>getReviewRosters</code>	POST <code>/api/evaluation/rosters/review</code>	Review/moderation rosters for logged-in reviewer
<code>getReviewRosterDetails</code>	POST <code>/api/evaluation/roster-details/review</code>	Scripts with both primary marks and reviewer's own marks
<code>getDashboardStats</code>	POST <code>/api/evaluation/dashboard-stats</code>	Different stats depending on role: admin sees org-wide stats; faculty sees personal stats

Function	Route	Description
getPublicationData	POST /api/osm/admin/rosters/results/publication-data	Combined OSM + CBT results — uses \$unionWith to merge offline and online results
getResultFilterOptions	POST /api/osm/admin/rosters/results/filter-options	Filter options (exams, courses, programs, schools, academic years, locations)
getEvaluatorAssessments	POST /api/eval/evaluator/assessments	Hierarchical view (Year → Program → Course) combining OSM rosters + CBT schedules for an evaluator
getScriptsForCourse	POST /api/eval/evaluator/course-scripts	Scripts for a specific OSM roster with evaluation status
getOrgSettings	POST /api/eval/org/settings	Organization settings including OSM flag
getCBTStudentsForSchedule	POST /api/eval/evaluator/cbt-students	Students in a CBT schedule with their submission and rubrics evaluation status

Key patterns:

- **saveEvaluationProgress** : Automatically transitions roster status:
 1. pending → in_progress on first save
 2. Counts completed scripts after each save; if completedCount >= roster.scriptCount → completed
- **getPublicationData** : Uses MongoDB \$unionWith to combine results from evaluation_results (OSM) and cbt_marks (CBT) into a single paginated, filterable result set. Final score = moderationScore ?? primaryScore for OSM; totalMarksObtained for CBT.
- **getEvaluatorAssessments** : Two-step cross-DB approach for CBT:
 1. Aggregate testScheduleModel (testplayer_db) to get schedule IDs + student counts
 2. Batch-fetch yearly_masters, courses, exam_informations (osm_db)
 3. Merge results into a unified hierarchical structure
- **getCBTStudentsForSchedule** : Since users_list in testplayer_db stores usernames (strings), the controller uses UserProgressModel (by schedule_Id) to get actual submitted students, then cross-references CBT_Marks for evaluation status.

9.5 icAssessmentsController.js

Handles Internal Continuous Assessment (ICA) view — showing all IC assessment tasks across all three modes.

Function	Route	Description
getICAssessments	POST /api/evaluation/data-entry/ic-assessments	Returns hierarchical IC assessment list (Year → Program → Course) across Manual Entry, OSM, and CBT Online modes

How it works:

1. Finds exam_type = "Internal Assessment" via examTypesModel
2. Gets faculty's assigned courses from facultyDirectoryModel
3. Queries **three sources** in parallel:
 - examInformationsModel for mode = "Paper Based + Data Entry" IC exams → **Manual Entry**
 - DistributionRosterModel for IC exams with evaluationType = "primary" → **OSM**
 - testScheduleModel for IC exam schedules with isRubricsPresent → **CBT Online**
4. Merges all three into a single hierarchical response

9.6 studentController.js

Student management module — allows admins to browse students with their enrollment, exam results, and scanned scripts.

Function	Route	Description
getFilterOptions	GET /api/osm/students/filters	Schools, programs, courses, batches, academic years, terms for filter UI
getHierarchicalData	GET /api/osm/students/hierarchical	Schools → Programs → Courses → Students with enrollment data
getStudentList	GET /api/osm/students/list	Paginated flat student list with search and filters; enriches each student with their enrolled courses/school/program
	GET	Full student profile: basic info + enrolled courses + CBT_Marks +

getStudentDetails Function	/api/osm/students/:studentId/details Route	Scanned_Scripts + summary stats Description
-------------------------------	---	--

10. Cross-Model Relationships (Implicit Refs)

These are relationships that exist in code but NOT as Mongoose `ref` declarations in the schemas. New joiners must be aware of these to understand how data flows.

From	Field	Points to	How resolved in code
users	orgId	org_details._id	Direct ObjectId query
users	role[]	org_roles._id	Direct ObjectId query
faculty_directories	user_details	users._id	\$lookup from users collection
faculty_evaluation_rosters	user_details[]	users._id	Direct query
student_enrolments	student_ids[]	users._id	User.find({ _id: { \$in: ... } })
distribution_rosters	testId	exam_informations.test_details._id	Matched via "test_details._id": roster.testId
evaluation_results	testId	exam_informations.test_details._id	Same pattern
marking_pallets	testId	exam_informations.test_details._id	Same pattern
exam_informations	org_id	org_details._id	Direct ObjectId query
scanned_scripts	test_details	exam_informations.test_details._id	Direct ObjectId comparison
answer_script_qrcodes_nu	candidate_id	users._id	Direct query
unique_script_code_mappings	candidateId, testId, examId, courseId	Various (no refs)	Direct queries
cbt_marks	candidateId	testplayer_db.User._id	Cross-DB; controllers join manually
cbt_marks	scriptId	testplayer_db.AnswerScript._id	Cross-DB; no direct lookup
cbt_marks	schedule_Id	testplayer_db.test_schedule_records._id	Cross-DB; no direct lookup
testSchedule (testplayer_db)	yearly_master_id	osm_db.yearly_masters._id	Two-step: fetch from osm_db separately
testSchedule (testplayer_db)	course_id	osm_db.courses._id	Two-step approach
testSchedule (testplayer_db)	exam_info_id	osm_db.exam_informations._id	Two-step approach
testSchedule (testplayer_db)	users_list[]	Strings (usernames/uniquenos)	User.find({ uniqueno: { \$in: users_list } })
scanned_scripts_nu (scanner_db)	test_details, schedule_details	ORGANIZATION_DB.test_schedule_records._id	Cross-DB; no direct lookup

11. Mermaid Diagram — Admin Models Only

```
erDiagram
    org_details {
        ObjectId _id PK
        String org_code UK
        String org_name UK
        String org_type
        ObjectId_arr privileges FK
        ObjectId org_parent
    }
    privileges {
        ObjectId _id PK
        String p_name
        String p_key_name
        Number p_id
    }
    org_roles {
        ObjectId _id PK
        String name
        ObjectId_arr privileges FK
        ObjectId org_id FK
    }
    org_admins {
        ObjectId _id PK
        String email UK
        Number role
        ObjectId org_id FK
        Boolean isSuperAdmin
        String uniqueno UK
    }
    users {
        ObjectId _id PK
        String email UK
        String username UK
        String uniqueno UK
        ObjectId orgId
        ObjectId_arr role
        String type
        String system
    }
    schools {
        ObjectId _id PK
        String name
        String location
        String code
        ObjectId org_id FK
    }
    programs {
        ObjectId _id PK
        String name
        String code
        String scheme
        ObjectId org_id FK
    }
    departments {
        ObjectId _id PK
        String name
        String code
        ObjectId org_id FK
    }
    courses {
        ObjectId _id PK
        String name
        String code
```



```
String code
ObjectId org_id FK
}
yearly_masters {
  ObjectId _id PK
  ObjectId school FK
  ObjectId program FK
  String batch
  String academic_year
  String term
  ObjectId_arr course_list FK
  ObjectId org_id FK
}
student_enrolments {
  ObjectId _id PK
  ObjectId academic_details FK
  ObjectId org_id FK
  ObjectId_arr student_ids
}
faculty_directories {
  ObjectId _id PK
  ObjectId academic_details FK
  ObjectId_arr role_details FK
  ObjectId course_details FK
  ObjectId user_details
  ObjectId org_id FK
}
exam_types {
  ObjectId _id PK
  String type
}
exam_sub_types {
  ObjectId _id PK
  String type
  ObjectId exam_type FK
}
test_types {
  ObjectId _id PK
  String type
}
exam_informations {
  ObjectId _id PK
  String exam_name
  ObjectId exam_type FK
  ObjectId exam_sub_type FK
  ObjectId yearly_master_details FK
  ObjectId_arr test_details
  ObjectId moderationRuleSetId FK
  ObjectId_arr multi_yearly_masters FK
  Boolean is_osm_enabled
}
exam_time_tables {
  ObjectId _id PK
  ObjectId academic_details FK
  ObjectId exam_id FK
  ObjectId_arr course_schedule_details
}
time_table_rosters {
  ObjectId _id PK
  ObjectId student_id
  ObjectId exam_id FK
  ObjectId yearly_master_id FK
  ObjectId time_table_id FK
}
marking_pallets {
  ObjectId _id PK
```

```

    ObjectId examId FK
    ObjectId courseId FK
    ObjectId testId
    ObjectId org_id FK
    String status
    Number totalMarks
    Object_arr questions
    ObjectId createdBy FK
    ObjectId frozenBy FK
}

sync_logs {
    ObjectId _id PK
    Object triggeredBy
    String status
    Object summary
}

moderation_rule_sets {
    ObjectId _id PK
    String name UK
    ObjectId org_id FK
    Object_arr rules
    Boolean isActive
}

org_details ||--o{ privileges : "has"
org_details ||--o{ org_roles : "defines"
org_roles ||--o{ privileges : "grants"
org_details ||--o{ org_admins : "administrated by"
org_details ||--o{ users : "has members"
org_details ||--o{ schools : "has campuses"
org_details ||--o{ programs : "offers"
org_details ||--o{ departments : "has"
org_details ||--o{ courses : "has"
org_details ||--o{ yearly_masters : "manages"
org_details ||--o{ moderation_rule_sets : "configures"
schools ||--o{ yearly_masters : "hosts batch"
programs ||--o{ yearly_masters : "runs in"
courses ||--o{ yearly_masters : "listed in course_list"
yearly_masters ||--o{ student_enrolments : "enrolls students"
users ||--o{ student_enrolments : "enrolled as student"
yearly_masters ||--o{ faculty_directories : "academic context"
courses ||--o{ faculty_directories : "assigned course"
users ||--o{ faculty_directories : "is faculty"
org_roles ||--o{ faculty_directories : "faculty roles"
exam_types ||--o{ exam_informations : "categorizes"
exam_types ||--o{ exam_sub_types : "has sub-types"
exam_sub_types ||--o{ exam_informations : "sub-categorizes"
yearly_masters ||--o{ exam_informations : "exam for batch"
test_types ||--o{ exam_informations : "test_details type"
courses ||--o{ exam_informations : "test_details course"
moderation_rule_sets ||--o{ exam_informations : "governs moderation"
yearly_masters ||--o{ exam_time_tables : "scheduled in"
exam_informations ||--o{ exam_time_tables : "scheduled as"
exam_informations ||--o{ time_table_rosters : "student rosters"
exam_time_tables ||--o{ time_table_rosters : "timetable"
users ||--o{ time_table_rosters : "student in roster"
yearly_masters ||--o{ time_table_rosters : "yearly master"
exam_informations ||--o{ marking_pallets : "has pallet"
courses ||--o{ marking_pallets : "per course"
users ||--o{ marking_pallets : "created or frozen by"
org_details ||--o{ marking_pallets : "owned by"

```

12. Mermaid Diagrams — Comprehensive System View

The system spans 4 databases and 3 evaluation modes — a single diagram would be unreadable. Below are **5 focused diagrams**, each covering one concern. Read them in order.

Arrow legend (applies to all diagrams below)

Arrow	Meaning
-->	Direct relationship — Mongoose <code>ref</code> or same-DB ObjectId field
- . "label" .->	Implicit / cross-DB reference — resolved in controller code only, no Mongoose <code>ref</code> declaration

12a — Org & Academic Hierarchy

Who owns what, and how the academic structure is organized inside `osm_db`.

```
flowchart LR
    subgraph osm["osm_db"]
        OD[org_details]
        PR[privileges]
        OR[org_roles]
        OA[org_admins]
        US[users]

        SC[schools]
        PG[programs]
        CO[courses]
        YM[yearly_masters]
        SE[student_enrolments]
        FD[faculty_directories]
    end

    OD --> PR
    OD --> OR
    OD --> OA
    OD --> US
    OD --> SC
    OD --> PG
    OD --> CO

    SC -->|school_details| YM
    PG -->|program| YM
    CO -->|course_list| YM

    YM -->|academic_details| SE
    US -. "student_ids[ ] — implicit ref" .-> SE

    YM -->|academic_details| FD
    CO -->|course_details| FD
    US -. "user_details — implicit ref" .-> FD
    OR -->|role check| FD

    style osm fill:#dbeeaf,stroke:#3b82f6,color:#1e3a8a
```

`yearly_masters` is the central pivot — it ties together the school, program, course list, and student batch for a given academic year.

12b — Exam Configuration & Marking Setup

How an exam is built and linked to marking schemes. The key insight: **there is no separate `tests` collection** — each entry in `exam_informations.test_details[]` auto-generates a Mongoose `_id`, and that `_id` is used as `testId` everywhere else in the system.

```

flowchart LR
    subgraph osm["osm_db"]
        YM[yearly_masters]
        ET[exam_types]
        ES[exam_sub_types]
        TT[test_types]
        EI[exam_informations]
        TD["test_details._id = testId"]
        ETB[exam_time_tables]
        TTR[time_table_rosters]
        MP[marking_pallets]
        MRS[moderation_rule_sets]
    end

    ET -->|exam_type| EI
    ES -->|exam_sub_type| EI
    YM -->|yearly_master_details| EI
    TT -. "test_type field inside each test_details entry" .-> EI

    EI -->|"embeds array – auto-generates _id per entry"| TD

    EI -. "examId" .-> MP
    TD -. "testId – the subdocument _id" .-> MP

    YM -->|yearly_master_details| ETB
    EI -->|exam_details| ETB
    ETB -->|generates roster entries| TTR

    MRS -. "admin links rule set to exam" .-> EI

    style osm fill:#dbeafe,stroke:#3b82f6,color:#1e3a8a

```

`marking_pallets` has a compound unique index on `(examId, courseId, testId)`. Once `status: frozen`, no further edits are allowed.

12c – OSM Evaluation Workflow

How scanned scripts flow from upload through primary marking, then through moderation. Both primary and moderation marks are stored in the same `evaluation_results` collection – distinguished by the `rosterId` they are keyed against.

```

flowchart LR
    subgraph osm["osm_db"]
        SS[Scanned_Scripts]
        MP[marking_pallets]
        US_E[users - evaluator]

        DR_P["distribution_rosters\nevaluationType: primary\nstatus: pending → in_progress → completed"]
        ER_P["evaluation_results\nentryType: osm - primary marks"]

        MRS[moderation_rule_sets]

        DR_M["distribution_rosters\nevaluationType: moderation"]
        ER_M["evaluation_results\nentryType: osm - moderation marks"]
    end

    SS -->|"assignedScripts[ ]"| DR_P
    US_E -->|"evaluatorId"| DR_P

    DR_P --> ER_P
    SS -. "scriptId - implicit ref" .-> ER_P
    MP -. "marking scheme" .-> ER_P
    US_E -. "evaluatorId - implicit ref" .-> ER_P

    ER_P -. "all scripts marked - roster auto-completes" .-> DR_P

    ER_P -->|"scores checked against rules"| MRS
    MRS -->|"qualifying scripts get new roster"| DR_M
    US_E -->|"moderatorId"| DR_M

    DR_M --> ER_M
    SS -. "scriptId - implicit ref" .-> ER_M

    style osm fill:#dbeafe,stroke:#3b82f6,color:#1e3a8a

```

When a faculty saves a mark, `saveEvaluationProgress` counts completed scripts for that roster. If `completedCount >= scriptCount`, the roster status auto-advances to `completed`.

12d — Data Entry & Manual Marks Workflow

How faculty manually enter marks for paper-based tests (no scanned scripts). The controller looks up the faculty's assigned courses via `faculty_directories`, then surfaces only exams whose test mode is `Paper Based + Data Entry`.

```

flowchart LR
    subgraph osm["osm_db"]
        US_F[users - faculty]
        FD[faculty_directories]
        EI["exam_informations\nmode: Paper Based + Data Entry"]
        TD["test_details._id = testId"]
        MP[marking_pallets]
        SE[student_enrolments]
        US_S[users - student]
        ER["evaluation_results\nentryType: data_entry\nkey: examId + testId + studentId"]
    end

    US_F -. "user_details - implicit ref" .-> FD
    FD -. "course_details - used to filter exams in controller" .-> EI
    EI -->|"embeds"| TD

    SE -. "student_ids[ ] - implicit ref" .-> US_S

    EI -. "examId" .-> ER
    TD -. "testId" .-> ER
    US_S -. "studentId - implicit ref" .-> ER
    US_F -. "evaluatorId - implicit ref" .-> ER
    MP -. "examId + testId - marking scheme" .-> ER

    style osm fill:#dbeafe,stroke:#3b82f6,color:#1e3a8a

```

IC Assessments (`getICAssessments`) uses the same faculty → course → exam lookup pattern but also adds OSM rosters and CBT schedules for the same courses into one merged response.

12e – CBT Cross-Database Flow

How the CBT platform data ties into the core OSM database. **All dashed arrows cross database boundaries** – MongoDB `$lookup` cannot span databases, so controllers use a two-step query: fetch IDs from Database A, then query Database B separately using `$in`.

```

flowchart LR
    subgraph osm["osm_db"]
        EI_O[exam_informations]
        YM_O[yearly_masters]
        CO_O[courses]
        CM["CBT_Marks\nisReportGenerated flag"]
    end

    subgraph tp["testplayer_db - CBT Platform"]
        TS["test_schedule_records\nusers_list: String[ ] - usernames"]
        QP["QuestionPapers\nisRubricsPresent"]
        AS[AnswerScripts]
        UP[UserProgress]
        CBT_U[users - CBT]
    end

    subgraph org["ORGANIZATION_DB"]
        TSR_O["test_schedule_records\nusers_list: ObjectId[ ] - student refs"]
        ASQR[answer_script_qrcodes_nu]
    end

    subgraph scan["scanner_db"]
        SSNU[Scanned_Scripts_NU]
    end

    TS -. "exam_info_id - points to osm_db" .-> EI_O
    TS -. "yearly_master_id - points to osm_db" .-> YM_O
    TS -. "course_id - points to osm_db" .-> CO_O

    QP --> AS
    QP --> UP
    CBT_U --> AS
    CBT_U --> UP
    TS --> AS
    TS --> UP

    CM -. "schedule_Id - points to testplayer_db" .-> TS
    CM -. "scriptId - points to testplayer_db" .-> AS
    CM -. "candidateId - points to testplayer_db" .-> CBT_U

    SSNU -. "schedule_details - points to ORGANIZATION_DB" .-> TSR_O
    ASQR --> TSR_O

    style osm fill:#dbeafe,stroke:#3b82f6,color:#1e3a8a
    style tp fill:#dcfce7,stroke:#16a34a,color:#14532d
    style org fill:#fef3c7,stroke:#f59e0b,color:#78350f
    style scan fill:#fce7f3,stroke:#db2777,color:#831843

```

Note: `test_schedule_records.users_list` stores `ObjectId[]` in `ORGANIZATION_DB` but `String[]` (usernames/uniquenos) in `testplayer_db`. This is a known schema discrepancy across the two databases.

Evaluation Flow Summary (Text)

1. SETUP

Admin creates exam_informations

- with test_details[] (one entry per course, each with a mode)
- links to yearly_masters (which student batch)
- optionally links to moderation_rule_sets

2. PRE-EVALUATION

Admin uploads synoptic PDFs to Azure → stored in exam_informations.test_details[].blueprint_details

Admin creates marking_pallets → defines max marks per question → frozen before evaluation starts

3. OSM FLOW

Scanner uploads Scanned_Scripts

- tagged with exam_details, course_details, test_details._id, candidateId

Admin runs Roster Engine → creates distribution_rosters (evaluationType = "primary")

- each roster: evaluatorId + assignedScripts[] + scriptCount

Faculty opens script → fetches PDF (Azure SAS URL) + marking pallet

Faculty saves marks → evaluation_results upserted

- roster auto-transitions: pending → in_progress → completed

4. DATA ENTRY FLOW

Faculty sees test list (getDataEntryTests, entryType = data_entry)

Faculty selects test → sees student list + marking pallet

Faculty saves marks → evaluation_results upserted with entryType = "data_entry"

5. MODERATION

Admin applies moderation rules (applyModerationAndCreateRosters)

- qualifying scripts get new distribution_rosters (evaluationType = "moderation")

Moderator evaluates → second set of evaluation_results

6. PUBLICATION

getPublicationData: merges evaluation_results (offline) + cbt_marks (online)

- finalScore = moderationScore ?? primaryScore (offline)

- finalScore = totalMarksObtained (online CBT)